

# ICS 2201 Object Oriented Programming II- JAVA

## Course Description/Outline

*In-depth study of Java. Development of large efficient and reusable systems: encapsulation, templates, references, constructors and destructors, operator overloading, memory management, exception handling, std templates, library, programming by contracts, configuration management, documentation and testing.*

Java Programming Language Notes

## What is Java?

Java is a popular programming language, created in 1995.

It is owned by Oracle, and more than **3 billion** devices run Java.

It is used for:

- Mobile applications (specially Android apps)
- Desktop applications
- Web applications
- Web servers and application servers
- Games
- Database connection

## Why Use Java?

- Java works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.)
- It is one of the most popular programming language in the world
- It is easy to learn and simple to use
- It is open-source and free
- It is secure, fast and powerful
- It has a huge community support (tens of millions of developers)
- Java is an objectoriented language which gives a clear structure to programs and allows code to be reused, lowering development costs

it is possible to write Java in an Integrated Development Environment, such as **IntelliJ IDEA( recommended for this course)**, Netbeans or Eclipse, which are particularly useful when managing larger collections of Java files

example

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

- Save the code in Notepad as "Main.java". Open Command Prompt (cmd.exe), navigate to the directory where you saved your file, and type "javac Main.java":
- `C:\Users\>javac Main.java`
- This will compile your code. If there are no errors in the code, the command prompt will take you to the next line. Now, type "java Main" to run the file
- `C:\Users\>java Main`
- Every line of code that runs in Java must be inside a `class`. In our example, we named the class **Main**. A class should always start with an uppercase first letter.
- **Note:** Java is case-sensitive: "MyClass" and "myclass" has different meaning.
- The name of the java file **must match** the class name. When saving the file, save it using the class name and add ".java" to the end of the filename.

## The main Method

The `main()` method is required and you will see it in every Java program:

```
public static void main(String[] args)
```

- Any code inside the `main()` method will be executed.
- For now, just remember that every Java program has a `class` name which must match the filename, and that every program must contain the `main()` method.

## System.out.println()

Inside the `main()` method, we can use the `println()` method to print a line of text to the screen:

```
public static void main(String[] args) {
```

```
System.out.println("Hello World");  
}
```

- Single-line comments start with two forward slashes (`//`).

```
System.out.println("Hello World"); // This is a comment
```

- Multi-line comments start with `/*` and ends with `*/`.
- Any text between `/*` and `*/` will be ignored by Java.

```
/* The code below will print the words Hello World  
to the screen, and it is amazing */  
System.out.println("Hello World");
```

## Java Variables

Variables are containers for storing data values.

In Java, there are different **types** of variables, for example:

- **String** - stores text, such as "Hello". String values are surrounded by double quotes
- **int** - stores integers (whole numbers), without decimals, such as 123 or -123
- **float** - stores floating point numbers, with decimals, such as 19.99 or -19.99
- **char** - stores single characters, such as 'a' or 'B'. Char values are surrounded by single quotes
- **boolean** - stores values with two states: true or false
- Where *type* is one of Java's types (such as **int** or **String**), and *variable* is the name of the variable (such as **x** or **name**). The **equal sign** is used to assign values to the variable.
- To create a variable that should store text, look at the following example:

```
String name = "John";  
System.out.println(name);
```

# Final Variables

However, you can add the `final` keyword if you don't want others (or yourself) to overwrite existing values (this will declare the variable as "final" or "constant", which means unchangeable and read-only):

```
final int myNum = 15;

myNum = 20; // will generate an error: cannot assign a value to a final variable
```

```
int myNum = 5;

float myFloatNum = 5.99f;

char myLetter = 'D';

boolean myBool = true;

String myText = "Hello";
```

# Display Variables

The `println()` method is often used to display variables.

To combine both text and a variable, use the `+` character:

```
String name = "John";

System.out.println("Hello " + name);
```

# Java Identifiers

All Java **variables** must be **identified** with **unique names**.

These unique names are called **identifiers**.

Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).

The general rules for constructing names for variables (unique identifiers) are:

- Names can contain letters, digits, underscores, and dollar signs
- Names must begin with a letter
- Names should start with a lowercase letter and it cannot contain whitespace
- Names can also begin with \$ and \_
- Names are case sensitive ("myVar" and "myvar" are different variables)
- Reserved words (like Java keywords, such as `int` or `boolean`) cannot be used as names

## Java Data Types

Data types are divided into two groups:

- Primitive data types - includes `byte`, `short`, `int`, `long`, `float`, `double`, `boolean` and `char`
- Non-primitive data types - such as [String](#), [Arrays](#) and [Classes](#)

## Primitive Data Types

A primitive data type specifies the size and type of variable values, and it has no additional methods.

There are eight primitive data types in Java:

| Data Type | Size    | Description                                                                       |
|-----------|---------|-----------------------------------------------------------------------------------|
| byte      | 1 byte  | Stores whole numbers from -128 to 127                                             |
| short     | 2 bytes | Stores whole numbers from -32,768 to 32,767                                       |
| int       | 4 bytes | Stores whole numbers from -2,147,483,648 to 2,147,483,647                         |
| long      | 8 bytes | Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 |
| float     | 4 bytes | Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits           |
| double    | 8 bytes | Stores fractional numbers. Sufficient for storing 15 decimal digits               |
| boolean   | 1 bit   | Stores true or false values                                                       |
| char      | 2 bytes | Stores a single character/letter or ASCII values                                  |

# Numbers

Primitive number types are divided into two groups:

**Integer types** stores whole numbers, positive or negative (such as 123 or -456), without decimals. Valid types are `byte`, `short`, `int` and `long`. Which type you should use, depends on the numeric value.

**Floating point types** represents numbers with a fractional part, containing one or more decimals. There are two types: `float` and `double`.

## Int

The `int` data type can store whole numbers from -2147483648 to 2147483647. In general, and in our tutorial, the `int` data type is the preferred data type when we create variables with a numeric value.

## Long

The `long` data type can store whole numbers from -9223372036854775808 to 9223372036854775807. This is used when `int` is not large enough to store the value. Note that you should end the value with an "L":

# Floating Point Types

You should use a floating point type whenever you need a number with a decimal, such as 9.99 or 3.14515.

## Float

The `float` data type can store fractional numbers from  $3.4e-038$  to  $3.4e+038$ . Note that you should end the value with an "f":

```
float myNum = 5.75f;  
System.out.println(myNum);
```

# Booleans

A boolean data type is declared with the `boolean` keyword and can only take the values `true` or `false`:

```
boolean isJavaFun = true;

boolean isFishTasty = false;

System.out.println(isJavaFun);    // Outputs true
System.out.println(isFishTasty);  // Outputs false
```

# Characters

The `char` data type is used to store a **single** character. The character must be surrounded by single quotes, like `'A'` or `'c'`:

```
char myGrade = 'B';

System.out.println(myGrade);
```

# Strings

The `String` data type is used to store a sequence of characters (text). String values must be surrounded by double quotes:

```
String greeting = "Hello World";

System.out.println(greeting);
```

# Non-Primitive Data Types

Non-primitive data types are called **reference types** because they refer to objects.

The main difference between **primitive** and **non-primitive** data types are:

- Primitive types are predefined (already defined) in Java. Non-primitive types are created by the programmer and is not defined by Java (except for `String`).

- Non-primitive types can be used to call methods to perform certain operations, while primitive types cannot.
- A primitive type has always a value, while non-primitive types can be `null`.
- A primitive type starts with a lowercase letter, while non-primitive types starts with an uppercase letter.
- The size of a primitive type depends on the data type, while non-primitive types have all the same size.

Examples of non-primitive types are [Strings](#), [Arrays](#), [Classes](#), [Interface](#), etc.

## Java Type Casting

Type casting is when you assign a value of one primitive data type to another type.

In Java, there are two types of casting:

- **Widening Casting** (automatically) - converting a smaller type to a larger type size  
`byte -> short -> char -> int -> long -> float -> double`
- **Narrowing Casting** (manually) - converting a larger type to a smaller size type  
`double -> float -> long -> int -> char -> short -> byte`

Java divides the operators into the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Bitwise operators

## Double

The `double` data type can store fractional numbers from  $1.7e-308$  to  $1.7e+308$ . Note that you should end the value with a "d":

```
double myNum = 19.99d;
System.out.println(myNum);
```

# Arithmetic Operators

Arithmetic operators are used to perform common mathematical operations.

| Operator | Name           | Description                            | Example          |
|----------|----------------|----------------------------------------|------------------|
| +        | Addition       | Adds together two values               | $x + y$          |
| -        | Subtraction    | Subtracts one value from another       | $x - y$          |
| *        | Multiplication | Multiplies two values                  | $x * y$          |
| /        | Division       | Divides one value by another           | $x / y$          |
| %        | Modulus        | Returns the division remainder         | $x \% y$         |
| ++       | Increment      | Increases the value of a variable by 1 | <code>++x</code> |
| --       | Decrement      | Decreases the value of a variable by 1 | <code>--x</code> |

A list of all assignment operators:

| Operator | Example                    | Same As                       |
|----------|----------------------------|-------------------------------|
| =        | <code>x = 5</code>         | <code>x = 5</code>            |
| +=       | <code>x += 3</code>        | <code>x = x + 3</code>        |
| -=       | <code>x -= 3</code>        | <code>x = x - 3</code>        |
| *=       | <code>x *= 3</code>        | <code>x = x * 3</code>        |
| /=       | <code>x /= 3</code>        | <code>x = x / 3</code>        |
| %=       | <code>x %= 3</code>        | <code>x = x % 3</code>        |
| &=       | <code>x &amp;= 3</code>    | <code>x = x &amp; 3</code>    |
| =        | <code>x  = 3</code>        | <code>x = x   3</code>        |
| ^=       | <code>x ^= 3</code>        | <code>x = x ^ 3</code>        |
| >>=      | <code>x &gt;&gt;= 3</code> | <code>x = x &gt;&gt; 3</code> |
| <<=      | <code>x &lt;&lt;= 3</code> | <code>x = x &lt;&lt; 3</code> |

| Operator | Name                     | Example |
|----------|--------------------------|---------|
| ==       | Equal to                 | x == y  |
| !=       | Not equal                | x != y  |
| >        | Greater than             | x > y   |
| <        | Less than                | x < y   |
| >=       | Greater than or equal to | x >= y  |
| <=       | Less than or equal to    | x <= y  |

## Java Logical Operators

Logical operators are used to determine the logic between variables or values:

| Operator | Name        | Description                                             | Example            |
|----------|-------------|---------------------------------------------------------|--------------------|
| &&       | Logical and | Returns true if both statements are true                | x < 5 && x < 10    |
|          | Logical or  | Returns true if one of the statements is true           | x < 5    x < 4     |
| !        | Logical not | Reverse the result, returns false if the result is true | !(x < 5 && x < 10) |

## String Length

A String in Java is actually an object, which contain methods that can perform certain operations on strings. For example, the length of a string can be found with the `length()` method:

```
String txt = "ABCDEFGHJKLMNOPQRSTUVWXYZ";
System.out.println("The length of the txt string is: " + txt.length());
```

There are many string methods available, for example `toUpperCase()` and `toLowerCase()`:

```
String txt = "Hello World";
System.out.println(txt.toUpperCase()); // Outputs "HELLO WORLD"
System.out.println(txt.toLowerCase()); // Outputs "hello world"
```

# Finding a Character in a String

The `indexOf()` method returns the **index** (the position) of the first occurrence of a specified text in a string (including whitespace):

```
String txt = "Please locate where 'locate' occurs!";  
System.out.println(txt.indexOf("locate")); // Outputs 7
```

## String Concatenation

The `+` operator can be used between strings to combine them. This is called **concatenation**:

```
String firstName = "John";  
String lastName = "Doe";  
System.out.println(firstName + " " + lastName);
```

You can also use the `concat()` method to concatenate two strings:

```
String firstName = "John ";  
String lastName = "Doe";  
System.out.println(firstName.concat(lastName));
```

## Special Characters

Because strings must be written within quotes, Java will misunderstand this string, and generate an error:

```
String txt = "We are the so-called "Vikings" from the north.";
```

The solution to avoid this problem, is to use the **backslash escape character**.

The backslash (`\`) escape character turns special characters into string characters:

| Escape character | Result | Description  |
|------------------|--------|--------------|
| \'               | '      | Single quote |
| \"               | "      | Double quote |
| \\               | \      | Backslash    |

Six other escape sequences are valid in Java:

| Code | Result          |
|------|-----------------|
| \n   | New Line        |
| \r   | Carriage Return |
| \t   | Tab             |
| \b   | Backspace       |
| \f   | Form Feed       |

## Math.max(x,y)

The `Math.max(x,y)` method can be used to find the highest value of x and y

## Math.sqrt(x)

The `Math.sqrt(x)` method returns the square root of x

## Math.abs(x)

The `Math.abs(x)` method returns the absolute (positive) value of

# Boolean Expression

A **Boolean expression** is a Java expression that returns a Boolean value: **true** or **false**.

You can use a comparison operator, such as the **greater than** (**>**) operator to find out if an expression (or a variable) is true:

## Syntax

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

## Syntax

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}
```

## Syntax

```
if (condition1) {  
    // block of code to be executed if condition1 is true  
} else if (condition2) {  
    // block of code to be executed if the condition1 is false and  
    condition2 is true  
} else {  
    // block of code to be executed if the condition1 is false and  
    condition2 is false  
}
```

# Short Hand If...Else (Ternary Operator)

There is also a short-hand if else, which is known as the **ternary operator** because it consists of three operands. It can be used to replace multiple lines of code with a single line. It is often used to replace simple if else statements

## Syntax

```
variable = (condition) ? expressionTrue : expressionFalse;
```

# Java Switch Statements

Use the `switch` statement to select one of many code blocks to be executed.

## Syntax

```
switch(expression) {  
    case x:  
        // code block  
        break;  
    case y:  
        // code block  
        break;  
    default:  
        // code block  
}
```

# The break Keyword

When Java reaches a `break` keyword, it breaks out of the switch block.

This will stop the execution of more code and case testing inside the block.

When a match is found, and the job is done, it's time for a break. There is no need for more testing.

## The default Keyword

The `default` keyword specifies some code to run if there is no case match:

## Java While Loop

The `while` loop loops through a block of code as long as a specified condition is `true`:

### Syntax

```
while (condition) {  
    // code block to be executed  
}
```

## The Do/While Loop

The `do/while` loop is a variant of the `while` loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

### Syntax

```
do {  
    // code block to be executed  
}  
while (condition);
```

### Syntax

```
for (statement 1; statement 2; statement 3) {  
    // code block to be executed
```

```
}  
  
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

## For-Each Loop

There is also a "**for-each**" loop, which is used exclusively to loop through elements in an **array**:

### Syntax

```
for (type variableName : arrayName) {  
    // code block to be executed  
}  
  
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};  
for (String i : cars) {  
    System.out.println(i);  
}
```

## Java Arrays

Arrays are used to store multiple values in a single variable, instead of declaring separate variables for each value.

To declare an array, define the variable type with **square brackets**:

```
String[] cars;
```

## Change an Array Element

To change the value of a specific element, refer to the index number:

## Example

```
cars[0] = "Opel";
```

## Example

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};

cars[0] = "Opel";

System.out.println(cars[0]);

// Now outputs Opel instead of Volvo
```

# Array Length

To find out how many elements an array has, use the `length` property:

## Example

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};

System.out.println(cars.length);

// Outputs 4
```

# Loop Through an Array

You can loop through the array elements with the `for` loop, and use the `length` property to specify how many times the loop should run.

The following example outputs all elements in the **cars** array:

## Example

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};

for (int i = 0; i < cars.length; i++) {

    System.out.println(cars[i]);

}
```

# Multidimensional Arrays

A multidimensional array is an array of arrays.

To create a two-dimensional array, add each array within its own set of **curly braces**:

## Example

```
int[][] myNumbers = { {1, 2, 3, 4}, {5, 6, 7} };
```

**myNumbers** is now an array with two arrays as its elements.

To access the elements of the **myNumbers** array, specify two indexes: one for the array, and one for the element inside that array. This example accesses the third element (2) in the second array (1) of myNumbers:

## Example

```
int[][] myNumbers = { {1, 2, 3, 4}, {5, 6, 7} };  
int x = myNumbers[1][2];  
System.out.println(x); // Outputs 7
```

## Java Methods

- A **method** is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a method.
- Methods are used to perform certain actions, and they are also known as **functions**.
- Why use methods? To reuse code: define the code once, and use it many times.
- A method must be declared within a class. It is defined with the name of the method, followed by parentheses **()**. Java provides some pre-defined

methods, such as `System.out.println()`, but you can also create your own methods to perform certain actions:

```
public class Main {  
  
    static void myMethod() {  
  
        // code to be executed  
  
    }  
  
}
```

- `myMethod()` is the name of the method
- `static` means that the method belongs to the Main class and not an object of the Main class.
- `void` means that this method does not have a return value
- To call a method in Java, write the method's name followed by two parentheses `()` and a semicolon;
- Inside `main`, call the `myMethod()` method:

```
public class Main {  
    static void myMethod() {  
        System.out.println("I just got executed!");  
    }  
  
    public static void main(String[] args) {  
        myMethod();  
    }  
}  
  
// Outputs "I just got executed!"
```

## Parameters and Arguments

Information can be passed to methods as parameter. Parameters act as variables inside the method.

Parameters are specified after the method name, inside the parentheses. You can add as many parameters as you want, just separate them with a comma.

The following example has a method that takes a `String` called **fname** as parameter. When the method is called, we pass along a first name, which is used inside the method to print the full name:

```

public class Main {

    static void myMethod(String fname) {

        System.out.println(fname + " Kamau");

    }

    public static void main(String[] args) {

        myMethod("Liam");

    }

}

// Liam Kamau

```

When a **parameter** is passed to the method, it is called an **argument**. So, from the example above: **fname** is a **parameter**, while **Lilian** is an **argument**.

## Multiple Parameters

You can have as many parameters as you like:

```

public class Main {

    static void myMethod(String fname, int age) {

        System.out.println(fname + " is " + age);

    }

    public static void main(String[] args) {

        myMethod("John", 4);

        myMethod("Jenny", 8);

        myMethod("Auma", 21);

    }

}

```

```
}
```

Note that when you are working with multiple parameters, the method call must have the same number of arguments as there are parameters, and the arguments must be passed in the same order.

## Return Values

The `void` keyword, used in the examples above, indicates that the method should not return a value. If you want the method to return a value, you can use a primitive data type (such as `int`, `char`, etc.) instead of `void`, and use the `return` keyword inside the method:

```
public class Main {  
    static float myMethod(float x) {  
        return 5.14 + x;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(myMethod(5.22));  
    }  
}  
  
// Outputs 10.36 (5.14 + 5.22)  
  
public class Main {  
  
    // Create a checkAge() method with an integer variable called age  
    static void checkAge(int age) {  
  
        // If age is less than 18, print "access denied"  
        if (age < 18) {
```

```

        System.out.println("Access denied - You are not old enough!");

        // If age is greater than, or equal to, 18, print "access granted"
    } else {
        System.out.println("Access granted - You are old enough!");
    }

}

public static void main(String[] args) {
    checkAge(20); // Call the checkAge method and pass along an age of 20
}
}

// Outputs "Access granted - You are old enough!"

```

## Method Overloading

With **method overloading**, multiple methods can have the same name with different parameters:

```

int myMethod(int x)

float myMethod(float x)

double myMethod(double x, double y)

```

Instead of defining two methods that should do the same thing, it is better to overload one.

In the example below, we overload the `plusMethod` method to work for both `int` and `double`:

```

static int plusMethod(int x, int y) {

```

```

    return x + y;
}

static double plusMethod(double x, double y) {
    return x + y;
}

public static void main(String[] args) {
    int myNum1 = plusMethod(8, 5);
    double myNum2 = plusMethod(4.3, 6.26);
    System.out.println("int: " + myNum1);
    System.out.println("double: " + myNum2);
}

```

## Java Scope

In Java, variables are only accessible inside the region they are created. This is called **scope**.

## Method Scope

Variables declared directly inside a method are available anywhere in the method following the line of code in which they were declared:

```

public class Main {
    public static void main(String[] args) {

        // Code here CANNOT use x
    }
}

```

```
int x = 100;

// Code here can use x

System.out.println(x);
}
}
```

## Block Scope

A block of code refers to all of the code between curly braces `{}`. Variables declared inside blocks of code are only accessible by the code between the curly braces, which follows the line in which the variable was declared:

```
public class Main {
    public static void main(String[] args) {

        // Code here CANNOT use x

        { // This is a block

            // Code here CANNOT use x

            int x = 100;
            // Code here CAN use x

            System.out.println(x);

        } // The block ends here
    }
}
```

```
// Code here CANNOT use x

}

}
```

## Java Recursion

Recursion is the technique of making a function call itself. This technique provides a way to break complicated problems down into simple problems which are easier to solve.

Recursion may be a bit difficult to understand. The best way to figure out how it works is to experiment with it.

## Recursion Example

Adding two numbers together is easy to do, but adding a range of numbers is more complicated. In the following example, recursion is used to add a range of numbers together by breaking it down into the simple task of adding two numbers:

Use recursion to add all of the numbers up to 100.

```
public class Main {

    public static void main(String[] args) {

        int result = sum(100);

        System.out.println(result);

    }

    public static int sum(int k) {

        if (k > 0) {

            return k + sum(k - 1);

        }

    }

}
```

```

    } else {
        return 0;
    }
}
}

```

## Example Explained

When the `sum()` function is called, it adds parameter `k` to the sum of all numbers smaller than `k` and returns the result. When `k` becomes 0, the function just returns 0. When running, the program follows these steps:

```

10 + sum(9)
10 + ( 9 + sum(8) )
10 + ( 9 + ( 8 + sum(7) ) )
...
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + sum(0)
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + 0

```

Since the function does not call itself when `k` is 0, the program stops there and returns the result.

## Halting Condition

Just as loops can run into the problem of infinite looping, recursive functions can run into the problem of infinite recursion. Infinite recursion is when the function never stops calling itself. Every recursive function should have a halting condition, which is the condition where the function stops calling itself. In the previous example, the halting condition is when the parameter `k` becomes 0.

It is helpful to see a variety of different examples to better understand the concept. In this example, the function adds a range of numbers between a start and an end. The halting condition for this recursive function is when **end** is not greater than **start**:

Use recursion to add all of the numbers between 5 to 10.

```

public class Main {
    public static void main(String[] args) {

```

```

int result = sum(5, 10);

System.out.println(result);
}

public static int sum(int start, int end) {
    if (end > start) {
        return end + sum(start, end - 1);
    } else {
        return end;
    }
}
}

```

## JAVA as an OOP language.

OOP stands for **Object-Oriented Programming**.

Procedural programming is about writing procedures or methods that perform operations on the data, while object-oriented programming is about creating objects that contain both data and methods.

Object-oriented programming has several advantages over procedural programming:

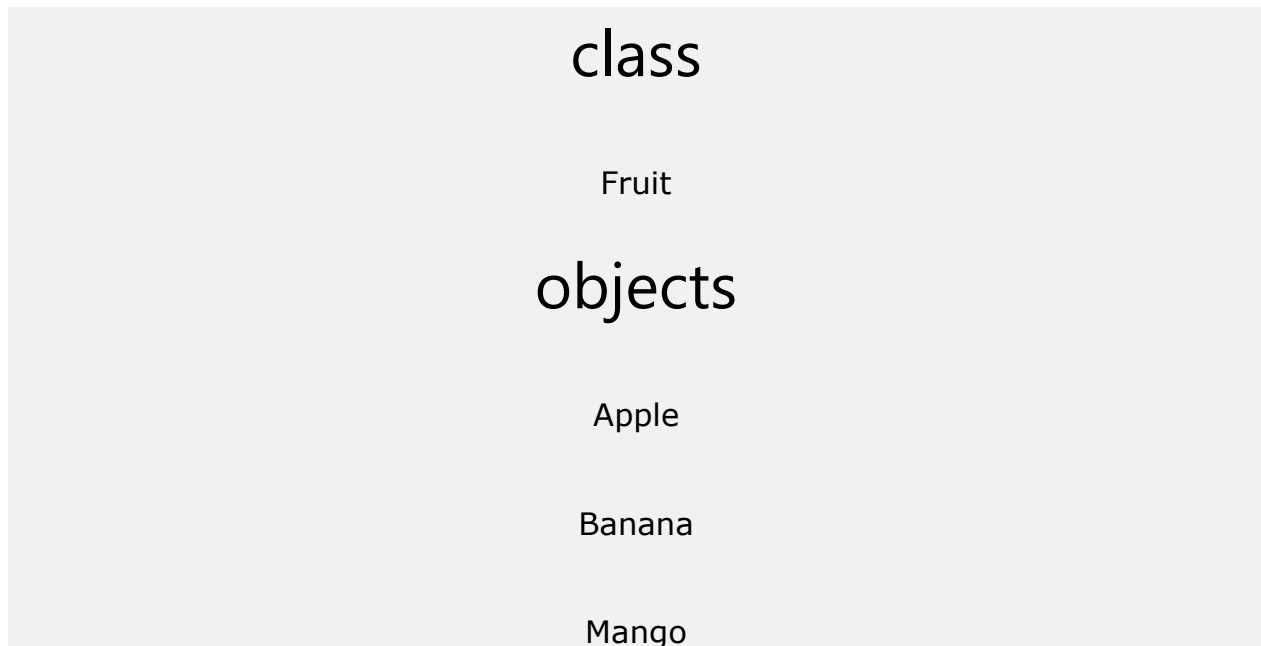
- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the Java code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

**Tip:** The "Don't Repeat Yourself" (DRY) principle is about reducing the repetition of code. You should extract out the codes that are common for the application, and place them at a single place and reuse them instead of repeating it.

# What are Classes and Objects?

Classes and objects are the two main aspects of object-oriented programming.

Look at the following illustration to see the difference between class and objects:



- So, a class is a template for objects, and an object is an instance of a class. When the individual objects are created, they inherit all the variables and methods from the class.
- Everything in Java is associated with classes and objects, along with its attributes and methods. For example: in real life, a car is an object. The car has **attributes**, such as weight and color, and **methods**, such as drive and brake.
- A Class is like an object constructor, or a "blueprint" for creating objects.

To create a class, use the keyword `class`:

```
public class Main {  
    int x = 5;  
}
```

In Java, an object is created from a class. We have already created the class named `Main`, so now we can use this to create objects.

To create an object of `Main`, specify the class name, followed by the object name, and use the keyword `new`:

Create an object called `"myObj"` and print the value of `x`:

```
public class Main {  
    int x = 5;  
  
    public static void main(String[] args) {  
        Main myObj = new Main();  
        System.out.println(myObj.x);  
    }  
}
```

## Using Multiple Classes

You can also create an object of a class and access it in another class. This is often used for better organization of classes (one class has all the attributes and methods, while the other class holds the `main()` method (code to be executed)).

Remember that the name of the java file should match the class name. In this example, we have created two files in the same directory/folder:

- `Main.java`
- `Second.java`

### ***Main.java***

```
public class Main {  
    int x = 5;  
}
```

## Second.java

```
class Second {  
  
    public static void main(String[] args) {  
  
        Main myObj = new Main();  
  
        System.out.println(myObj.x);  
  
    }  
  
}
```

- the term "variable" , means actually an **attribute** of the class. Or you could say that class attributes are variables within a class. Another term for class attributes is **fields**.

## Accessing Attributes

You can access attributes by creating an object of the class, and by using the dot syntax (.):

The following example will create an object of the `Main` class, with the name `myObj`. We use the `x` attribute on the object to print its value:

Create an object called "`myObj`" and print the value of `x`:

```
public class Main {  
  
    int x = 5;  
  
  
    public static void main(String[] args) {  
  
        Main myObj = new Main();  
  
        System.out.println(myObj.x);  
  
    }  
  
}
```

Change the value of `x` to 25:

```

public class Main {

    int x = 10;

    public static void main(String[] args) {

        Main myObj = new Main();

        myObj.x = 25; // x is now 25

        System.out.println(myObj.x);

    }

}

```

## Multiple Objects

If you create multiple objects of one class, you can change the attribute values in one object, without affecting the attribute values in the other:

Change the value of `x` to 25 in `myObj2`, and leave `x` in `myObj1` unchanged:

```

public class Main {

    int x = 5;

    public static void main(String[] args) {

        Main myObj1 = new Main(); // Object 1

        Main myObj2 = new Main(); // Object 2

        myObj2.x = 25;

        System.out.println(myObj1.x); // Outputs 5

        System.out.println(myObj2.x); // Outputs 25

    }

}

```

# Multiple Attributes

You can specify as many attributes as you want:

```
public class Main {  
    String fname = "John";  
    String lname = "Doe";  
    int age = 24;  
  
    public static void main(String[] args) {  
        Main myObj = new Main();  
        System.out.println("Name: " + myObj.fname + " " + myObj.lname);  
        System.out.println("Age: " + myObj.age);  
    }  
}
```

- `myMethod()` prints a text (the action), when it is **called**. To call a method, write the method's name followed by two parentheses `()` and a semicolon;

Inside `main`, call `myMethod()`:

```
public class Main {  
    static void myMethod() {  
        System.out.println(" Jambo Kenya!");  
    }  
  
    public static void main(String[] args) {  
        myMethod();  
    }  
}
```

```
}
```

```
// Outputs "Jambo Kenya!"
```

## Static vs. Non-Static

You will often see Java programs that have either `static` or `public` attributes and methods.

In the example above, we created a `static` method, which means that it can be accessed without creating an object of the class, unlike `public`, which can only be accessed by objects:

An example to demonstrate the differences between `static` and `public methods`:

```
public class Main {  
  
    // Static method  
  
    static void myStaticMethod() {  
  
        System.out.println("Static methods can be called without creating  
objects");  
  
    }  
  
  
    // Public method  
  
    public void myPublicMethod() {  
  
        System.out.println("Public methods must be called by creating  
objects");  
  
    }  
  
  
    // Main method  
  
    public static void main(String[] args) {  
  
        myStaticMethod(); // Call the static method  
  
    }  
}
```

```
// myPublicMethod(); This would compile an error

Main myObj = new Main(); // Create an object of Main
myObj.myPublicMethod(); // Call the public method on the object
}
}
```

## Example

Create a Car object named `myCar`. Call the `fullThrottle()` and `speed()` methods on the `myCar` object, and run the program:

```
// Create a Main class
public class Main {

    // Create a fullThrottle() method
    public void fullThrottle() {
        System.out.println("The car is moving very fast!");
    }

    // Create a speed() method and add a parameter
    public void speed(int maxSpeed) {
        System.out.println("Max speed is: " + maxSpeed);
    }

    // Inside main, call the methods on the myCar object
    public static void main(String[] args) {
        Main myCar = new Main(); // Create a myCar object
```

```
myCar.fullThrottle();    // Call the fullThrottle() method
myCar.speed(260);        // Call the speed() method
}
}
```

## Example explained

- 1) We created a custom `Main` class with the `class` keyword.
- 2) We created the `fullThrottle()` and `speed()` methods in the `Main` class.
- 3) The `fullThrottle()` method and the `speed()` method will print out some text, when they are called.
- 4) The `speed()` method accepts an `int` parameter called `maxSpeed` - we will use this in **8**).
- 5) In order to use the `Main` class and its methods, we need to create an **object** of the `Main` Class.
- 6) Then, go to the `main()` method, which you know by now is a built-in Java method that runs your program (any code inside main is executed).
- 7) By using the `new` keyword we created an object with the name `myCar`.
- 8) Then, we call the `fullThrottle()` and `speed()` methods on the `myCar` object, and run the program using the name of the object (`myCar`), followed by a dot (`.`), followed by the name of the method (`fullThrottle()`; and `speed(200)`;). Notice that we add an `int` parameter of **260** inside the `speed()` method.

## Very Important Point(VIP)...

The dot (`.`) is used to access the object's attributes and methods.

To call a method in Java, write the method name followed by a set of parentheses (`()`), followed by a semicolon (`;`).

A class must have a matching filename (`Main` and **`Main.java`**).

# Using Multiple Classes

Like we specified in the [Classes chapter](#), it is a good practice to create an object of a class and access it in another class.

Remember that the name of the java file should match the class name. In this example, we have created two files in the same directory:

- Main.java
- Second.java

## ***Main.java***

```
public class Main {  
    public void fullThrottle() {  
        System.out.println("The car is going as fast as it can!");  
    }  
  
    public void speed(int maxSpeed) {  
        System.out.println("Max speed is: " + maxSpeed);  
    }  
}
```

## ***Second.java***

```
class Second {  
    public static void main(String[] args) {  
        Main myCar = new Main();    // Create a myCar object  
        myCar.fullThrottle();        // Call the fullThrottle() method  
        myCar.speed(200);            // Call the speed() method  
    }  
}
```

# Java Constructors

A constructor in Java is a **special method** that is used to initialize objects. The constructor is called when an object of a class is created.

Note that the constructor name must **match the class name**, and it cannot have a **return type** (like `void`).

Also note that the constructor is called when the object is created.

All classes have constructors by default: if you do not create a class constructor yourself, Java creates one for you. However, then you are not able to set initial values for object attributes.

- Constructors can also take parameters, which is used to initialize attributes.

```
public class Main {  
    int modelYear;  
    String modelName;  
  
    public Main(int year, String name) {  
        modelYear = year;  
        modelName = name;  
    }  
  
    public static void main(String[] args) {  
        Main myCar = new Main(1969, "Mustang");  
        System.out.println(myCar.modelYear + " " + myCar.modelName);  
    }  
}
```

## Modifiers for classes

|                      |                                                                                                                 |
|----------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>public</code>  | The class is accessible by any other class                                                                      |
| <code>default</code> | The class is only accessible by classes in the same package.<br>This is used when you don't specify a modifier. |

**attributes, methods and constructors**, you can use the one of the following modifiers:

|                        |                                                                                                     |
|------------------------|-----------------------------------------------------------------------------------------------------|
| <code>public</code>    | The code is accessible for all classes                                                              |
| <code>private</code>   | The code is only accessible within the declared class                                               |
| <code>default</code>   | The code is only accessible in the same package.<br>This is used when you don't specify a modifier. |
| <code>protected</code> | The code is accessible in the same package and subclasses                                           |

## Non-Access Modifiers

For **classes**, you can use either `final` or `abstract`:

|                       |                                                |
|-----------------------|------------------------------------------------|
| <code>final</code>    | The class cannot be inherited by other classes |
| <code>abstract</code> | The class cannot be used to create objects     |

For **attributes and methods**, you can use the one of the following:

|                           |                                                                                                                                                                                                                                    |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>final</code>        | Attributes and methods cannot be overridden/modified                                                                                                                                                                               |
| <code>static</code>       | Attributes and methods belongs to the class, rather than an object                                                                                                                                                                 |
| <code>abstract</code>     | <p>Can only be used in an abstract class, and can only be used on methods.</p> <p>The method does not have a body, for example <code>abstract void run();</code>.</p> <p>The body is provided by the subclass (inherited from)</p> |
| <code>transient</code>    | Attributes and methods are skipped when serializing the object containing them                                                                                                                                                     |
| <code>synchronized</code> | Methods can only be accessed by one thread at a time                                                                                                                                                                               |
| <code>volatile</code>     | The value of an attribute is not cached thread-locally, and is always read from the "main memory"                                                                                                                                  |

- If you don't want the ability to override existing attribute values, declare attributes as `final`

```
public class Main {  
  
    final int x = 10;  
  
    final double PI = 3.14;  
  
    public static void main(String[] args) {  
  
        Main myObj = new Main();  
  
        myObj.x = 50; // will generate an error: cannot assign a value to a  
        final variable  
  
        myObj.PI = 25; // will generate an error: cannot assign a value to a  
        final variable  
  
        System.out.println(myObj.x);  
  
    }  
}
```

A `static` method means that it can be accessed without creating an object of the class, unlike `public`:

```
public class Main {  
  
    // Static method  
  
    static void myStaticMethod() {  
  
        System.out.println("Static methods can be called without creating  
objects");  
  
    }  
  
    // Public method  
  
    public void myPublicMethod() {
```

```

        System.out.println("Public methods must be called by creating
objects");
    }

    // Main method
    public static void main(String[ ] args) {
        myStaticMethod(); // Call the static method
        // myPublicMethod(); This would output an error

        Main myObj = new Main(); // Create an object of Main
        myObj.myPublicMethod(); // Call the public method
    }
}

```

## Abstract

An **abstract** method belongs to an **abstract** class, and it does not have a body. The body is provided by the subclass:

```

// Code from filename: Main.java

// abstract class
abstract class Main {

    public String fname = "John";

    public int age = 24;

    public abstract void study(); // abstract method
}

// Subclass (inherit from Main)
class Student extends Main {

```

```

    public int graduationYear = 2018;

    public void study() { // the body of the abstract method is provided
here
        System.out.println("Studying all day long");
    }
}

// End code from filename: Main.java


// Code from filename: Second.java
class Second {

    public static void main(String[] args) {

        // create an object of the Student class (which inherits attributes
and methods from Main)

        Student myObj = new Student();

        System.out.println("Name: " + myObj.fname);

        System.out.println("Age: " + myObj.age);

        System.out.println("Graduation Year: " + myObj.graduationYear);

        myObj.study(); // call abstract method
    }
}

```